04 栈和队列

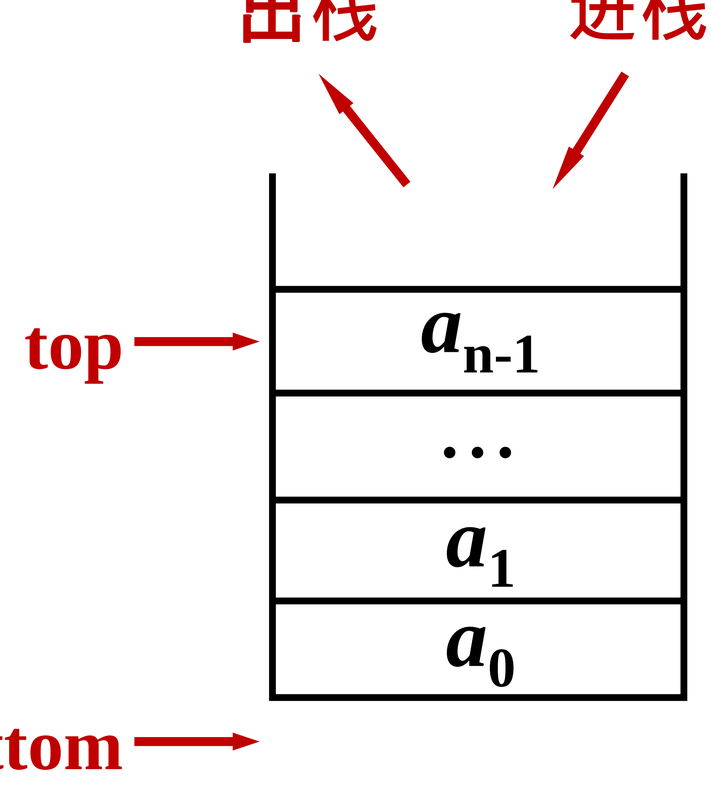
- 栈
- 队列

04 栈和队列

- 栈
- 队列

栈(Stack)

- 栈是只允许在一端进行插入和删除的线性表
- 允许插入和删除的一端叫栈顶(top)，另一端叫栈底(bottom)
- 特点：后进先出(LIFO)



栈的抽象数据类型

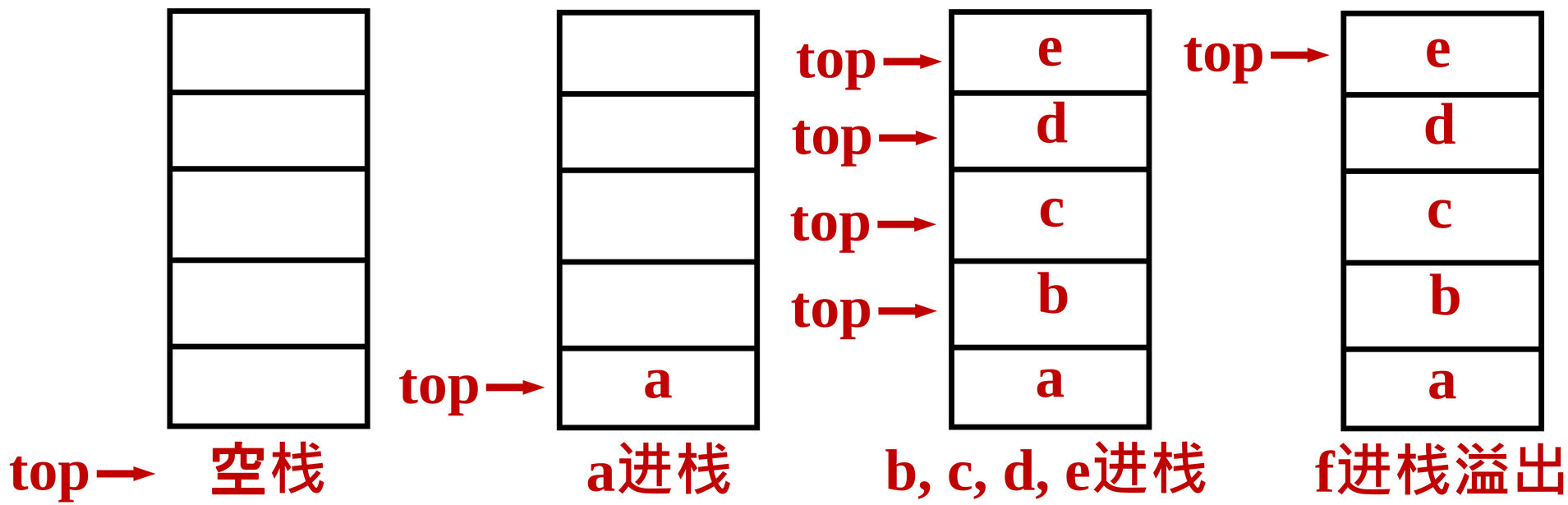
```
template <class Type> class Stack {
public:
    Stack( );  // 构造函数
    void Push(Type x);  // 进栈
    int Pop(Type &x);  // 出栈
    int GetTop(Type &x);  // 取栈顶元素
    void MakeEmpty ( );  // 置空栈
    int IsEmpty( );  // 判栈空否
    int IsFull( );  // 判栈满否
    int GetSize( );  // 返回栈中元素个数
};
```

栈的数组存储表示：顺序栈

```
template <class Type> class Stack {
private:
    int top;  // 栈顶指针(数组下标指针)
    Type *elements;  // 栈元素数组头指针
    int maxSize;  // 栈最大容量
 public:
    Stack (int sz = 10);  // 构造函数
    ~Stack ( ) { delete [ ] elements; }
    void Push (Type x);  // 进栈
    int Pop (Type& x);  // 出栈
    int GetTop (Type& x);  // 取栈顶元素
    void MakeEmpty ( ) { top = -1; }  // 置空栈
    int IsEmpty ( ) { return top == -1; }
    int IsFull ( ) { return top == maxSize -  1; }
    int GetSize ( ) { return top + 1; }
}
```

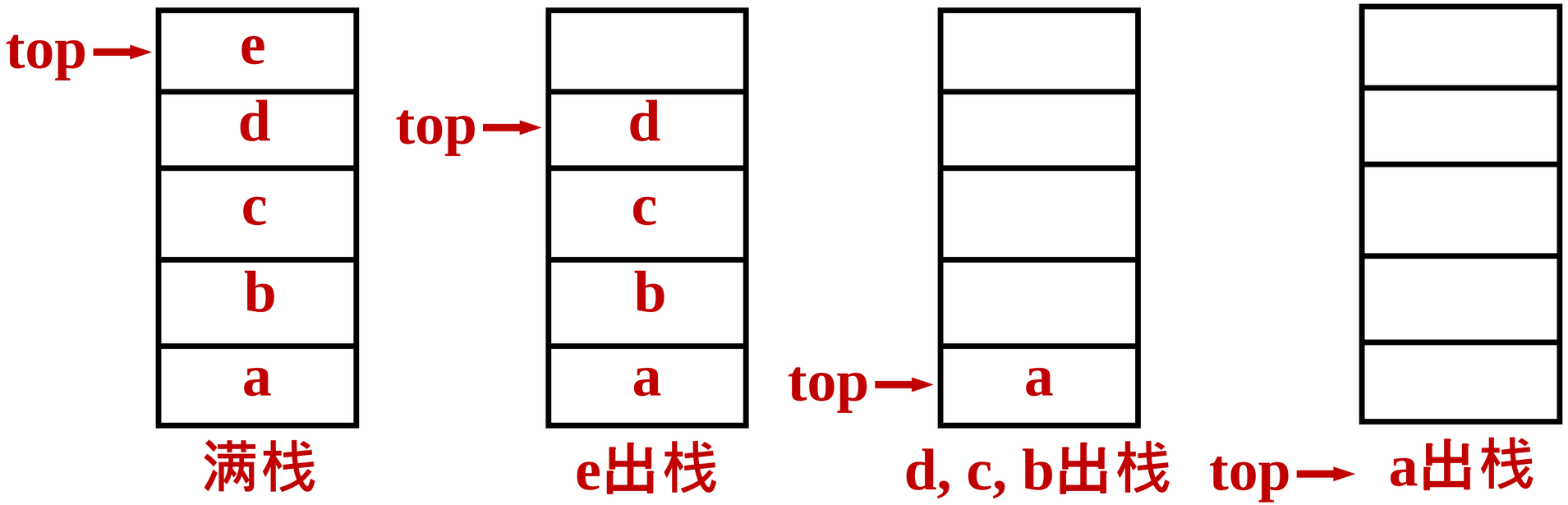
进栈操作

- 栈的最后允许存放位置为maxSize-1，需判断栈是否已满
- 如果栈顶指针top==maxSize-1(栈已满)，则栈中所有位置均已使用，若有新元素进栈，发生溢出，进行溢出处理
- 如果top<maxSize-1，则先让栈顶指针加1，指到当前可加入新元素的位置，再按栈顶指针所指位置将新元素插入，该元素成为新的栈顶元素



出栈操作

- 极端情况出现在栈底，需判断栈是否为空
- 如果top==-1(栈已空)，则执行栈空处理，栈空处理不是出错处理，而是该栈的算法结束时需要执行的处理
- 如果top>=0，则栈顶指针减1，即栈顶退回到次栈顶位置



进栈、出栈与取栈顶元素实现

```
template <class Type> void Stack <Type> :: Push(Type x) {
    if (IsFull( ))  OverflowProcess( );  // 溢出处理
    elements[++top] = x;  // 加入新元素
}
```

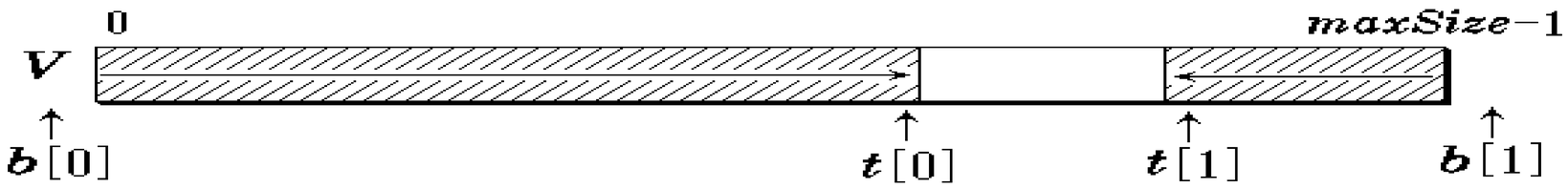
```
template <class Type> int Stack <Type> :: Pop(Type &x) {
    if (IsEmpty( ))  return 0;  // 栈空不退栈
    x = elements[top--];  // 退出栈顶元素
    return 1;
}
```

```
template <class Type> int Stack <Type> :: GetTop(Type &x) {
    if (IsEmpty( ))  return 0;
    x = elements[top];  // 取出栈顶元素
    return 1;
}
```

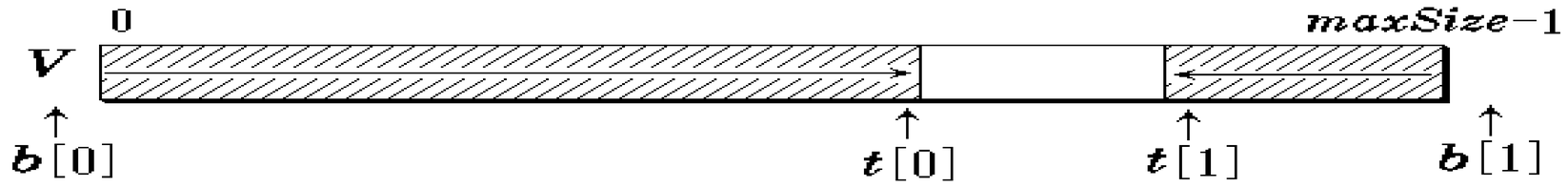
两个栈共用一个栈空间

- 为避免栈满时溢出，需定义一个足够大的栈空间
  - 如果栈中实际只有几个元素，是一种空间浪费

- 当需要两个栈时，可定义一个足够大的栈空间
  - 两个栈的栈顶都向中间伸展，直到两个栈的栈顶相遇，发生溢出
  - 一个栈可能进栈元素多而体积大些，另一个则可能小些
  - 两个栈共用一个栈空间，互相调剂，灵活性强



两个栈共用一个栈空间



- 两个栈共享一个数组空间**V[maxSize]**
- 设立栈顶指针数组**t[2]**和栈底指针数组**b[2]**
  - **t[i]**和**b[i]**分别指示**第i栈**的栈顶与栈底
  - 初始时，**t[0] = b[0] = -1**，**t[1] = b[1] = maxSize**
- 栈满条件：**t[0] + 1 == t[1]**，即栈顶指针相遇
- 栈空条件：**t[0] = b[0]**或**t[1] = b[1]**，即栈顶指针退到栈底

两个栈共用一个栈空间

```
template <class Type> class DoubleStack {
    private:
        int maxSize;
        Type * space;
        int t[2], b[2];  // 栈顶和栈底指针数组
    public:
        DoubleStack(int sz) {
            maxSize = sz;
            t[0] = b[0] = -1;
            t[1] = b[1] = maxSize;
            space = new Type[maxSize];
        }
        ……
```

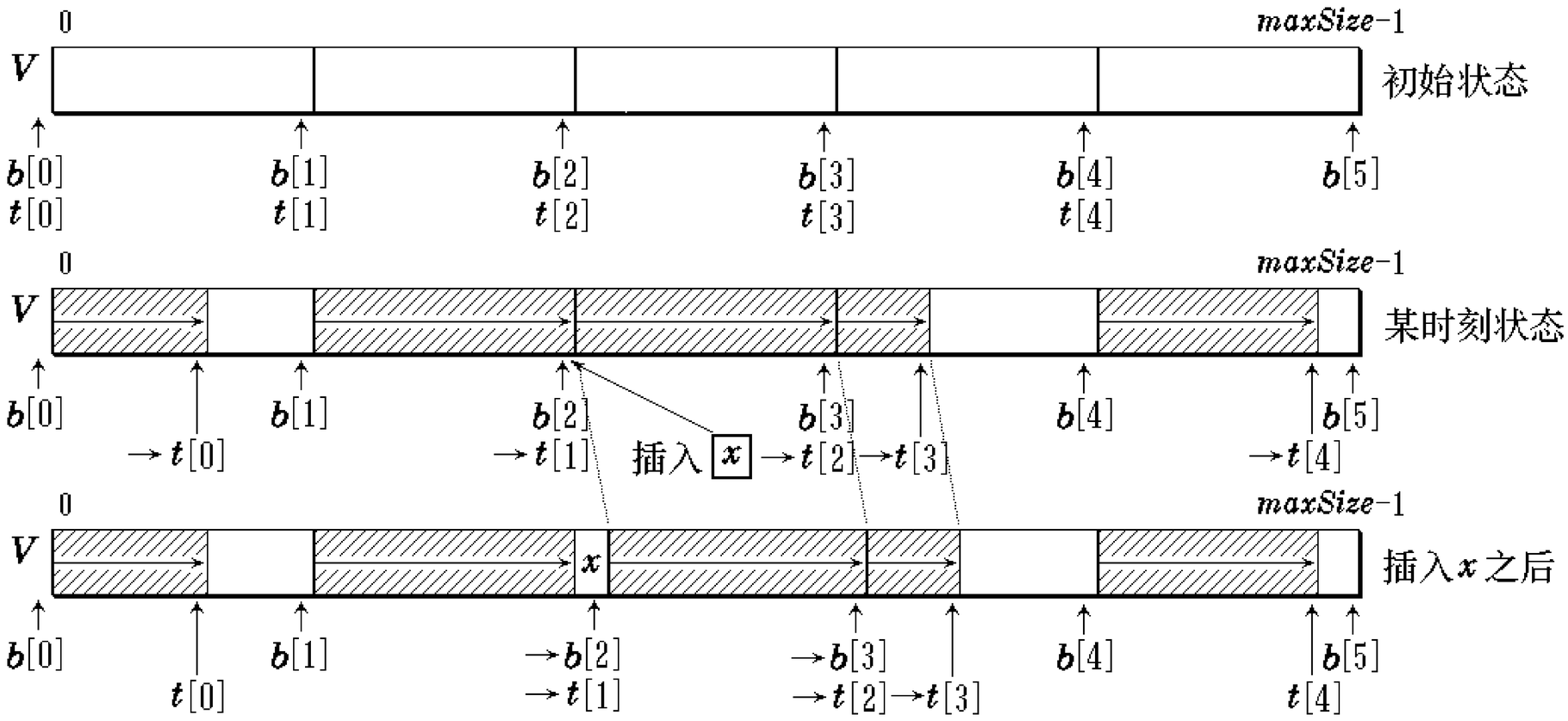
两个栈共用一个栈空间

```
int Push (Type x, int i) {
    if ( t[0] + 1 == t[1] )  return 0;  // 栈满栈顶指针相遇
    if ( i == 0 )  t[0]++;
    else  t[1]--;
    space[t[i]] = x;
    return 1;
  }
  int Pop (Type &x, int i) {
    if ( t[i] == b[i] )  return 0;  // 栈空：t[0] = b[0]或t[1] = b[1]
    x = space[t[i]];
    if ( i == 0 )  t[0]--;
    else t[1]++;
    return 1;
  }
};
```

多栈处理

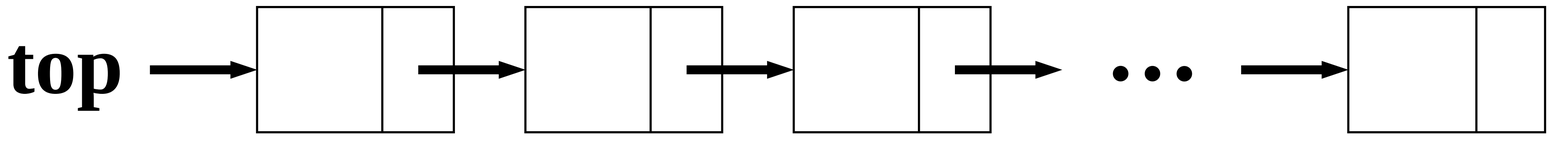
- 当程序中同时存在几个栈，问题较复杂
  - 各个栈所需空间在运行中动态变化
  - 若给几个栈分配同样大小的空间，有的栈膨胀得快，很快就溢出
  - 其它栈可能还有许多空闲的空间，需调整栈空间，防止溢出
  - 在插入时元素的移动量非常大，因而代价较高
  - 特别是当整个存储空间即将充满时，问题更加严重
- 栈浮动技术
  - n栈共享一个数组空间V[m]
  - 设立栈顶指针数组t[n+1]和栈底指针数组b[n+1]
  - t[i]和b[i]分别指示第i个栈的栈顶与栈底
  - b[n]作为控制量，指到数组最高下标
  - 各栈初始分配空间$s = \lfloor m/n \rfloor$
  - 指针初始值t[0]=b[0]=-1, b[n]=m-1, t[i]=b[i]=b[i-1]+s, i=1, …, n-1

多栈处理



插入新元素时的栈满处理 StackFull ( )

栈的链表存储表示：链式栈



- 链式栈(Linked Stack)无栈满问题，空间可扩充
- 插入与删除仅在栈顶处执行
- 链式栈的栈顶在链头
- 适合于多栈操作

链式栈的类定义

```
template <class Type> class Stack;
template <class Type> class StackNode {
    friend class Stack<Type>;
    private:
        Type data;  // 结点数据
        StackNode <Type> *link;  // 结点链指针
    public:
        StackNode ( Type d = 0, StackNode<Type> * next = NULL ) :
            data (d), link (next) { }
};
```

链式栈的类定义

```
template <class Type> class Stack {
    private:
        StackNode <Type> *top;  // 栈顶指针
    public:
        Stack ( ) : top ( NULL ) { }
        ~Stack ( );
        void Push (Type x);  // 进栈
        int Pop (Type& x);  // 出栈
        int GetTop (Type& x);  // 取栈顶元素
        void MakeEmpty ( );  // 实现与~Stack( )相同
        int IsEmpty ( ) { return top == NULL; }
}
```

链式栈的类定义

```
template <class Type> Stack<Type> :: ~Stack ( ) {
    StackNode<Type> *p;
    while ( top != NULL ) {  // 逐结点回收
        p = top;  top = top->link;
        delete p;
    }
}

template <class Type> void Stack<Type> :: Push (Type x) {
    // 新结点链入*top之前, 并成为新栈顶
    top = new StackNode<Type> (x, top);
}
```

链式栈的类定义

```
template <class Type> int Stack<Type> :: Pop (Type& x) {
    if ( IsEmpty ( ) ) return 0;
    StackNode<Type> *p = top;
    x = p->data;  top = top->link;  // 修改栈顶
    delete p;  // 释放
    return 1;
}

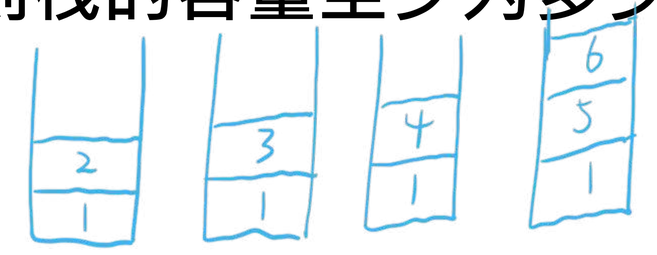
template <class Type> int Stack<Type>:: GetTop (Type& x) {
    if ( IsEmpty ( ) ) return 0;
    x = top->data;
    return 1;
}
```

练习1

- 设栈的进栈序列为1, 2, …, n，若出栈的第一个元素为n，则第i个出栈元素是什么？
- 若出栈的第一个元素为n，则由栈顶至栈底顺序存放的必然是n, n-1, …, 2, 1，故第i个出栈的元素为n-i+1

- 设栈的进栈序列为1, 2, …, n，若出栈的第一个元素为i，则第j个出栈元素是什么？
- 若i第一个出栈时，1, 2, …, i-1都在栈内，之后可能有i之后的元素进栈，究竟第 j个出栈的元素是哪个，不一定

- 设元素s_1, s_2, s_3, s_4, s_5, s_6依次进栈，若出栈顺序为s_2, s_3, s_4, s_6, s_5, s_1，则栈的容量至少为多少？
- 至少为3



栈应用：括号匹配

- 在(a*(b+c)-d)中，位置1和位置4有左括号，分别与位置11和位置8的右括号匹配
- 在(a+b))(中，位置6的右括号没有可匹配的左括号，位置7的左括号没有可匹配的右括号
- 可以观察到：从左向右扫描一个字符串，每一个右括号与最近遇到的未匹配的左括号相匹配
- 括号匹配的基本思路：从左向右扫描一个字符串，遇到左括号则进栈，遇到右括号就把栈顶的左括号出栈

栈应用：表达式求值

- 一个表达式由**操作数**、**操作符**和**分界符**组成

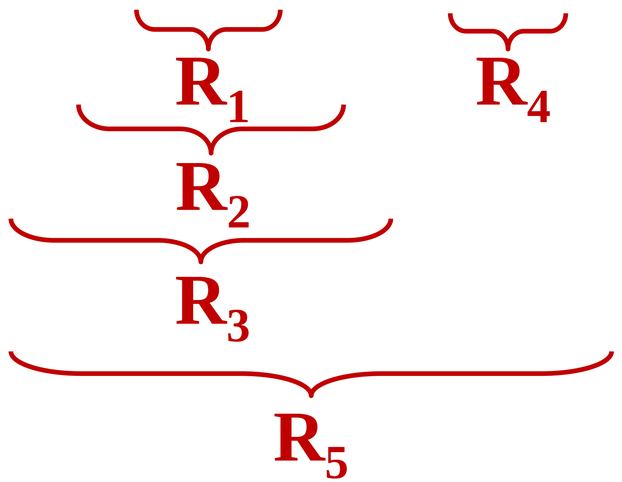
- 算术表达式有三种表示
  - **中缀(infix)表示：<操作数> <操作符> <操作数>， 如A+B；**
  - **前缀(prefix)表示：<操作符> <操作数> <操作数>， 如+AB；**
  - **后缀(postfix)表示：<操作数> <操作数> <操作符>， 如AB+；**

中缀表达式

- 中缀表达式示例：A+B*(C-D)-E/F
- 中缀表达式中，相邻两个操作符的计算次序依赖于操作符的优先级和括号
  - 优先级高的先计算
  - 优先级相同的自左向右计算
  - 括号从最内层开始计算
- 对于编译程序来说，一般不使用中缀表达式，而使用后缀表达式(较常用)和前缀表达式
- 对应的后缀表达式：ABCD-*+EF/-

后缀表达式的计算

- 在后缀表达式的计算顺序中已隐含了加括号的优先次序，括号在后缀表达式中不出现
- 从左向右顺序地扫描表达式，并用一个栈暂存扫描到的操作数或计算结果
- 计算示例：A B C D - * + E F / -



后缀表达式的计算

- 顺序扫描表达式的每一项，根据它的类型做如下操作
  - 若该项是**操作数**，则将其进栈
  - 若该项是**操作符<op>**，则连续出栈两个操作数**Y和X**，形成运算指令**X<op>Y**，并将计算结果重新进栈
- **当表达式的所有项都扫描并处理完后，栈顶存放的就是最后的计算结果**

A B C D - * + E F / -

中缀表示：A+B*(C-D)-E/F

步	输入	类型	动作	栈内容
1			置空栈	空
2	A	操作数	进栈	A
3	B	操作数	进栈	AB
4	C	操作数	进栈	ABC
5	D	操作数	进栈	ABCD
6	-	操作符	D、C出栈，计算C-D，结果R_1进栈	ABR_1
7	*	操作符	R_1、B出栈，计算B*R_1，结果R_2进栈	AR_2
8	+	操作符	R_2、A出栈，计算A+R_2，结果R_3进栈	R_3
9	E	操作数	进栈	R_3E
10	F	操作数	进栈	R_3EF
11	/	操作符	F、E出栈，计算E/F，结果R_4进栈	R_3R_4
12	-	操作符	R_4、R_3出栈，计算R_3-R_4，结果R_5进栈	R_5

Expression的类定义

```
class Expression {
    private:
        char * pos_e;  // 后缀表达式
        int * v;  // a, b, …, z的变量值
    public:
        Expression (char *posExpression, int *val) {
            pos_e = posExpression;
            v = val;
        }
        ~Expression ( ) {}
        int evaluate (int * p_y);  // 求值结果存在p_y所指的变量中
}
```

Expression的类定义

```
int Expression::evaluate(int *p_y) {
    int i, j, k, x, y, z;
    Stack<int> s(100);
    char c;
    i = 0;  c = pos_e[0];
    while( c != '\0' ) {
        if ( islower(c) )  s.push(v[c-'a']);
        else switch (c) {
            case '+' :    s.pop(&x);      s.pop(&y);      s.push(y+x);       break;
            case '-' :    s.pop(&x);      s.pop(&y);      s.push(y-x);       break;
            case '*':     s.pop(&x);      s.pop(&y);      s.push(y*x);       break;
            case '/':     s.pop(&x);      s.pop(&y);      s.push(y/x);       break;
            default: return 0;
        }
        c = pos_e[++i];
    }
    s.pop(p_y);        if(s.isEmpty()) return 1;     else return 0;
}
```

i指向待处理的字符

v[0]存a的值，v[1]存b的值，…

用栈将中缀表示转为后缀表示

- 使用栈可将表达式的中缀表示转换成它的后缀表示，为了实现这种转换，需要考虑各操作符的优先级
- 各个算术操作符的优先级

操作符	;	(	*, /	+, -	)
isp(栈内)	0	1	5	3	6
icp(栈外)	0	6	4	2	1

- isp叫做栈内优先级(in stack priority)
- icp叫做栈外优先级(in coming priority)

isp > icp
从左到右计算

- 操作符优先级相等的情况只出现在括号配对或栈底的‘;’与表达式最后的‘;’配对时

用栈将中缀表示转为后缀表示

- **操作符栈初始化，将结束符‘;’进栈，然后读入中缀表达式字符流的首字符ch**
- **重复执行以下步骤，直到ch = ‘;’，同时栈顶的操作符也是‘;’，停止循环**
  - **若ch是操作数则直接输出，读入下一个字符ch**
  - **若ch是操作符，判断ch的icp和位于栈顶的操作符op的isp**
    - 若 icp (ch) > isp (op)，则ch进栈，读入下一个字符ch
    - 若 icp (ch) < isp (op)，则op出栈并输出
    - 若 icp (ch) == isp (op)，则出栈但不输出，若出栈的是‘(’则读入下一个字符ch
- **算法结束，输出序列即为所需的后缀表达式**

A+B*(C-D)-E/F;

步	输入	栈内容	语义	输出	动作
1		;			栈初始化
2	A	;		A	操作数A输出，读字符
3	+	;+	+ > ;		操作符+进栈，读字符
4	B	;+		B	操作数B输出，读字符
5	*	;+*	* > +		操作符*进栈，读字符
6	(	;+*(	(> *		操作符(进栈，读字符
7	C	;+*(		C	操作数C输出，读字符
8	-	;+*(-	- > (		操作符-进栈，读字符
9	D	;+*(-		D	操作数D输出，读字符
10	)	;+*(	) < -	-	操作符-出栈输出

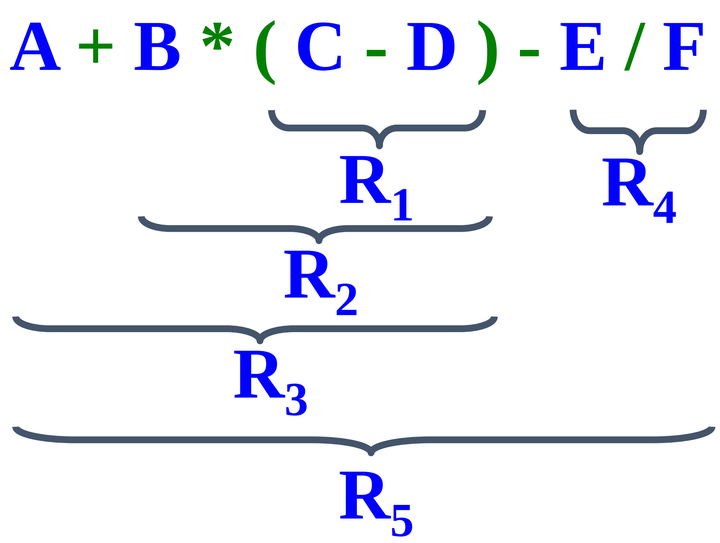
A+B*(C-D)-E/F;

步	输入	栈内容	语义	输出	动作
11		;+*	) = (		操作符(出栈不输出，读字符
12	-	;+	- < *	*	操作符*出栈输出
13		;	- < +	+	操作符+出栈输出
14		;-	- > ;		操作符-进栈，读字符
15	E	;-		E	操作数E输出，读字符
16	/	;-/	/ > -		操作符/进栈，读字符
17	F	;-/		F	操作数F输出，读字符
18	;	;-	; < /	/	操作符/出栈输出
19		;	; < -	-	操作符-出栈输出
20		;	; = ;		;配对，结束

用栈将中缀表示转为后缀表示

```
void postfix ( expression e ) {
    stack<char> s;  char ch, op;
    s.Push ( ';' );  cin.Get ( ch );
    while ( !s.IsEmpty( ) && ch != ';' )
        if ( isdigit ( ch ) ) {  //数字
            cout << ch;  cin.Get ( ch );  //输出;读入下一字符
        } else {
            if ( isp ( s.GetTop( ) ) < icp ( ch ) ) {  //栈外优先级高
                s.Push ( ch );  cin.Get ( ch );  //进栈;读入下一字符
            } else if ( isp ( s.GetTop( ) ) > icp ( ch ) ) {  //栈内优先级高
                op = s.GetTop ( );  s.Pop( );  cout << op;  //出栈并输出
            } else {  //优先级相同
                op = s.GetTop ( );  s.Pop( );  //出栈
                if ( op == '(' )  cin.Get ( ch );  //若出栈的是'(',则读入下一字符
            }
        }
}
```

中缀表达式求值



- 使用两个栈，操作符栈OPTR(Operator)，操作数栈OPND(Operand)
- 为了实现这种计算，需要考虑各操作符的优先级

中缀表达式求值

- **建立OPTR栈和OPND栈，并在OPTR栈进栈‘;’**
- **从头扫描中缀表达式，读一字符ch**
- **当ch != ‘;’或OPTR栈的栈顶 != ‘;’时，执行以下步骤；否则结束算法，此时在OPND栈的栈顶即运算结果**
  - **若ch是操作数，进OPND栈，从中缀表达式读下一字符ch**
  - **若ch是操作符，比较ch和OPTR的栈顶op的优先级**
    - 若icp(ch) > isp(op)，则ch进OPTR栈，从中缀表达式读下一字符ch
    - 若icp(ch) < isp(op)，则从OPND栈退出a2和a1，从OPTR栈退出op，形成运算指令 (a1) op (a2)，结果进OPND栈
    - 若icp(ch) == isp(op)且ch == ‘)’，则从OPTR栈退出‘(’，并从中缀表达式读下一字符ch

A+B*(C-D)-E/F;

步	输入	OPND	OPTR	语义	动作
1			;		栈初始化
2	A	A	;		操作数A进栈，读字符
3	+	A	;+	+ > ;	操作符+进栈，读字符
4	B	AB	;+		操作数B进栈，读字符
5	*	AB	;+*	* > +	操作符*进栈，读字符
6	(	AB	;+*(	(> *	操作符(进栈，读字符
7	C	ABC	;+*(		操作数C进栈，读字符
8	-	ABC	;+*(-	- > (	操作符-进栈，读字符
9	D	ABCD	;+*(-		操作数D进栈，读字符
10	)	ABR$_1$	;+*(	) < -	D、C、-出栈，计算C-D，结果R$_1$进栈

A+B*(C-D)-E/F;

步	输入	OPND	OPTR	语义	动作
11		ABR_1	;+*	) = (	操作符(出栈，读字符
12	-	AR_2	;+	- < *	R_1、B、*出栈，计算B*R_1，结果R_2进栈
13		R_3	;	- < +	R_2、A、+出栈，计算A+R_2，结果R_3进栈
14		R_3	;-	- > ;	操作符-进栈，读字符
15	E	R_3E	;-		操作数E进栈，读字符
16	/	R_3E	;-/	/ > -	操作符/进栈，读字符
17	F	R_3EF	;-/		操作数F进栈，读字符
18	;	R_3R_4	;-	; < /	F、E、/出栈，计算E/F，结果R_4进栈
19		R_5	;	; < -	R_4、R_3、-出栈，计算R_3-R_4，结果R_5进栈
20		R_5	;		;配对，结束，结果为R_5

中缀表达式求值

```
void InFixRun ( ) {
    stack<char> OPTR, OPND;  char ch, op;
    Push ( OPTR, ';' );  // 栈初始化
    getchar ( ch );  // 读一个字符
    op = ';' ;  //保存栈顶元素
    while ( ch != ';' || op != ';' ) {
        if ( isdigit ( ch ) ) { Push ( OPND, ch);  getchar( ch ); }
        else {  // 是操作符，比较优先级
            GetTop ( OPTR, op );  // 读一个操作符
            if ( icp( ch ) > isp( op ) ) { Push ( OPTR, ch );  getchar ( ch ); }
            else if ( icp ( ch ) < isp ( op ) ) { Pop( OPTR, op ); DoOperator ( OPND, op ); }
            else if ( ch == ')' ) { Pop( OPTR, op );  getchar ( ch ); }
        }
    }
}
```

栈应用：迷宫问题

- 迷宫问题：给定的迷宫中找出一条从入口到出口的路径

- 迷宫可用二维数组maze[m+2][n+2]表示
  - maze[i][j]为1表示该位置是墙壁，不通
  - maze[i][j]为0表示该位置是通路
  - $1 \le i \le m, 1 \le j \le n$
  - maze[1][0]为入口，maze[m][n+1]为出口
  - 数组的第0行和第m+1行、第0列和第n+1列是迷宫的围墙

用二维数组表示的迷宫

1 1 1 1 1 1 1 1 1 1

入口 0 0 0 1 0 0 1 1 0 1

1 1 0 0 0 1 1 0 1 1

1 0 1 1 1 0 0 1 1 1

1 0 1 0 1 1 1 0 0 0 出口

1 1 1 1 1 1 1 1 1 1

在迷宫中任一时刻的位置可用数组下标i与j来表示

可能的前进方向

西北 [*i*-1][*j*-1]	北 [*i*-1][*j*]	东北 [*i*-1][*j*+1]
西[*i*][*j*-1]	*X* [*i*][*j*]	[*i*][*j*+1] 东
[*i*+1][*j*-1] 西南	[*i*+1][*j*] 南	[*i*+1][*j*+1] 东南

从maze[i][j]出发，可能的前进方向有8个

前进方向表

move[*q*].d	*move*[*q*].*a*	*move*[*q*].*b*
北	-1	0
东北	-1	1
东	0	1
东南	1	1
南	1	0
西南	1	-1
西	0	-1
西北	-1	-1

在maze[i][j]时，若向东北方向走，下一相邻位置maze[g][h]为：

g = i + move[1].a = i - 1; h = j + move[1].b = j + 1;

迷宫问题的思路

- 用栈存储在试探过程中所走过的路径
  - 一旦需要回退，可以根据栈中记录进行回溯，尝试其它允许方向
  - 如果栈空，则表示已回退到最初出发位置
- 栈中数据用三元组表示<x, y, d>， 即在(x, y)位置沿d方向到达下一位置
- 标志矩阵防止走重复路
  - 标志矩阵mark[m+2][n+2]，其中所有元素都初始化为0
  - mark[i][j]为1表示该位置已走过
  - mark[i][j]为0表示该位置尚未走过

迷宫问题的算法

```
#define MAX 50
typedef struct element {int x, y, d; }  Element;  // 路径记录
typedef struct move {int a, b; }  MOVE;  // 前进方向
Stack <Element> s(MAX*MAX);
class Maze {
    private:
        int maze[MAX][MAX];  // 迷宫矩阵
        int mark[MAX][MAX];  // 标志矩阵
        int m;
        int n;
        MOVE mv[8];  // 前进方向表
    public:
        ……
        int getmazepath ( );
}
```

迷宫问题的算法

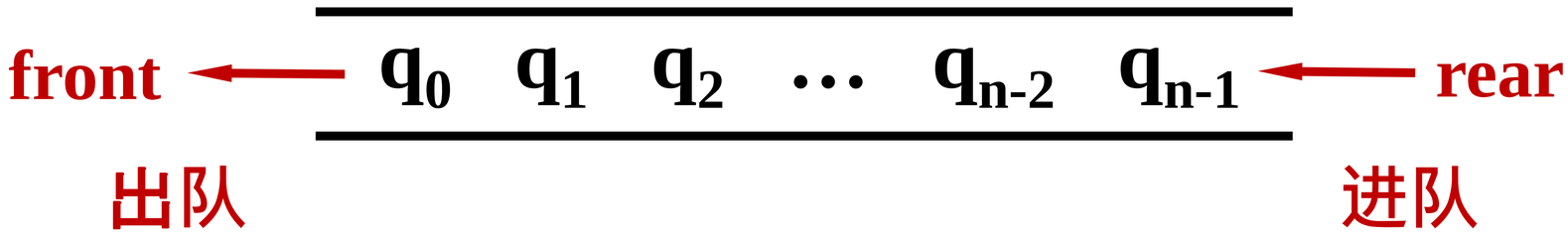
```
int getmazepath ( ) {
    int i, j, k, g, h;  Element current;
    if( maze[1][0] != 0 || maze[m][n+1] != 0 ) return ERROR;  // 出入口错误
    current.x = 1;  current.y = 0;  current.d = -1;  // 在入口，未走
    s.push(current);  mark[1][0] = 1;
    while( !s.isEmpty() )  {
        s.pop(&current);  i = current.x;  j = current.y;  k = current.d;
        while( k < 7 ) {  // 找下一个可走的方向
            g = i + mv[++k].a;  h = j + mv[k].b;
            if( g == m && h == n + 1) { printPath(i, j);  return OK; }  // 到达出口
            if( maze[g][h] == 0 && mark[g][h] == 0)  {  // 可走且未走过
                mark[g][h] = 1;  current.x = i;  current.y = j;  current.d = k;  s.push(current);
                i = g;  j = h;  k = -1;
            }
        }
    }
    return ERROR;
}
```

04 栈和队列

- 栈
- 队列

队列(Queue)

- 队列是只允许在一端删除，在另一端插入的线性表
- 允许删除的一端叫队头(front)，允许插入的一端叫队尾(rear)
- 特点：先进先出(FIFO)



队列的抽象数据类型

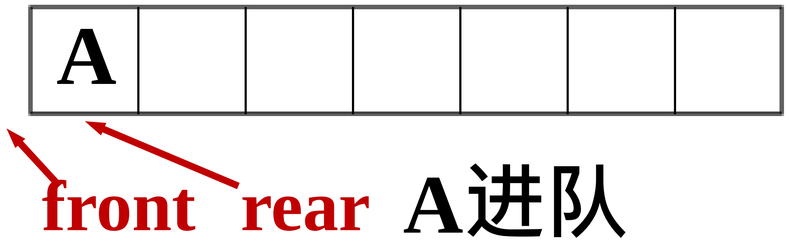
```
template <class Type> class Queue {
public:
    Queue (int = 10);  // 构造函数
    void EnQueue (Type x);  // 进队
    int DeQueue (Type& x);  // 出队
    int GetFront (Type& x);  // 取队头元素
    void MakeEmpty ( );  // 置空队列
    int IsEmpty ( ) const ;  // 判队列空否
    int IsFull ( ) const;  // 判队列满否
}
```

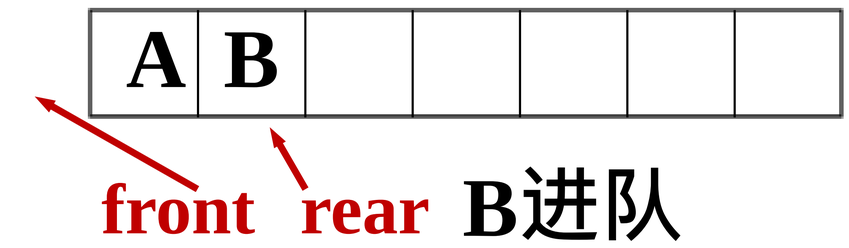
队列的数组存储表示：顺序队列

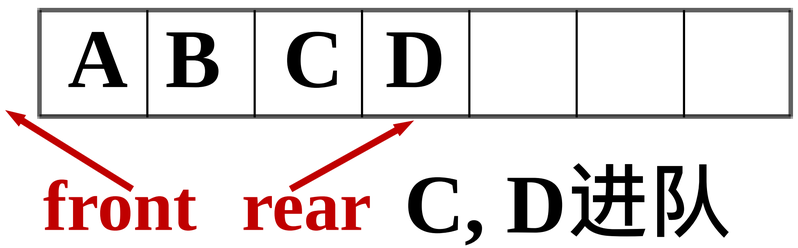
```
template <class Type> class Queue {
    private:
        int rear, front;  // 队尾，队头指针
        Type *elements;  // 队列元素数组
        int size;  // 最大元素个数
    public:
        Queue (int sz = 10);
        ~Queue ( ) { delete [ ] elements; }
        void EnQueue (Type x);  // 进队
        int DeQueue (Type& x);  // 出队
        int GetFront (Type& x);  // 取队头元素
        ……
}
```

队列的进队和出队

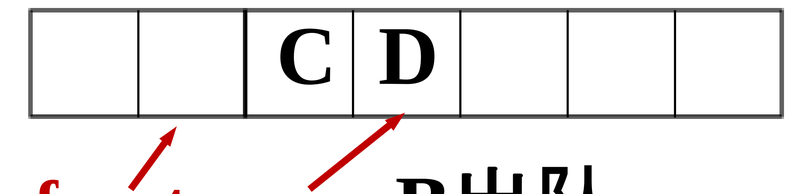

front rear 空队列


A
front rear A进队


A B
front rear B进队


A B C D
front rear C, D进队


B C D
front rear A出队


C D
front rear B出队


C D E F G
front rear E, F, G进队


C D E F G
front rear H进队，溢出

队列的进队和出队原则

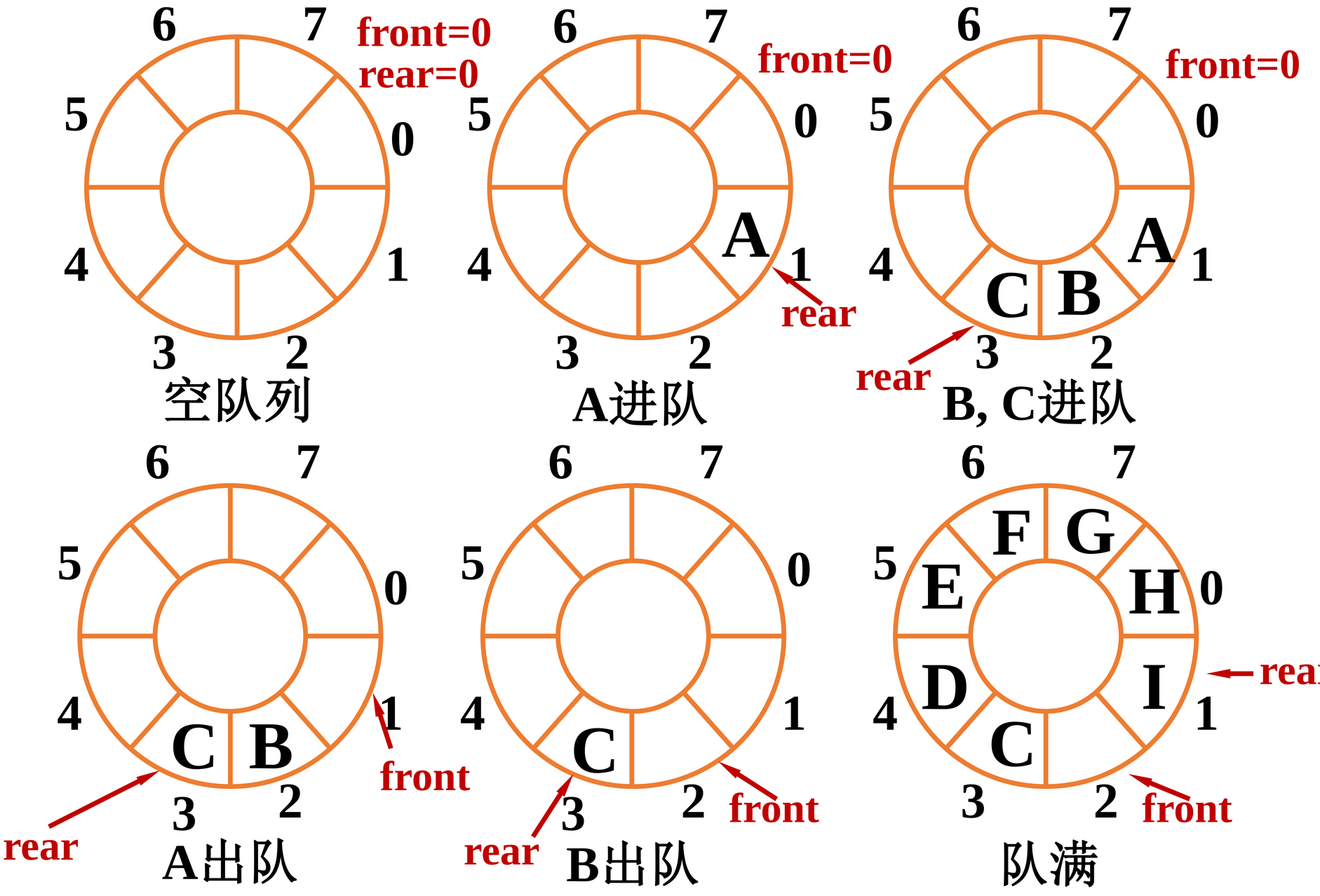
- 进队时，队尾指针先进一，即rear = rear + 1，再将新元素按rear指示位置加入，队尾指针指示实际队尾位置
- 出队时，队头指针先进一，即front = front + 1，再将下标为front的元素取出，队头指针指示实际队头的前一位置
- 队空时，满足front == rear，再出队将进行队空处理
- 队满时，满足rear == size-1，再进队将溢出，但可能是假溢出，因为前端可能有空间

- 解决假溢出的办法之一：将队列元素存放数组首尾相接，形成循环(环形)队列

循环队列(Circular Queue)

- 队列存放数组从逻辑上被当作首尾相接的表处理
- 队头、队尾指针进1时可从size -1直接进到0，可用程序语言的取模(余数)运算实现
- 队头指针进1：front = (front+1)%size
- 队尾指针进1：rear = (rear+1)%size
- 队列初始化：front = rear = 0
- 队空条件：front == rear
- 队满条件：(rear+1)%size == front

循环队列的进队和出队


6
7
front=0
rear=0
5
0
4
1
3
2
空队列
A
rear
front=0
A进队
C
B
rear
front
A出队
F
G
E
H
D
I
C
front
B, C进队
队满

队满说明

- 在到达队满状态时，实际还有一个空闲存储单元
- 如果想利用该单元，则队满和队空时都满足front==rear
- 为了区别，就需要加一个标志tag判断队列是满还是空
  - 最近一次执行是EnQueue，则tag==1；是DeQueue，则tag==0。
  - 当front==rear时，若tag==1，则队满；若tag==0，则队空
- 这增加了计算工作，在频繁使用的队列中一般不使用

循环队列的操作定义

```
template <class Type> void Queue <Type> :: MakeEmpty ( ) {
    front = rear = 0;
}

template <class Type> int Queue <Type> :: IsEmpty ( ) {
    return front == rear;
}

template <class Type> int Queue <Type> :: IsFull ( ) {
    return (rear+1)%size == front;
}
```

循环队列的操作定义

```
template <class Type> Queue <Type> ::
Queue (int sz) : front (0), rear (0), size (sz) {
    elements = new Type[size];
    assert ( elements != NULL );
}

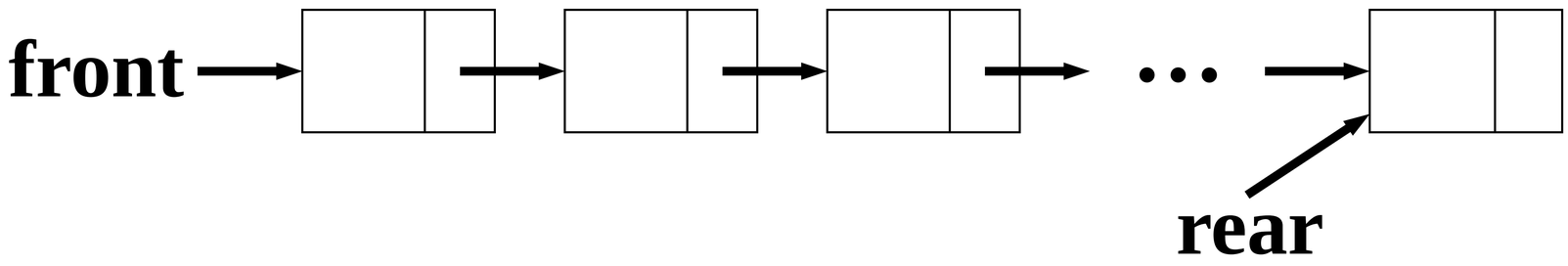
template <class Type> void Queue <Type> ::
EnQueue (Type x) {
    assert ( !IsFull ( ) );
    rear = (rear+1) % size;
    elements[rear] = x;
}
```

循环队列的操作定义

```
template <class Type> int Queue<Type> ::
DeQueue (Type& x) {
    if ( IsEmpty ( ) ) return 0;
    front = (front+1) % size;
    x = elements[front];  return 1;
}

template <class Type> int Queue<Type> ::
GetFront (Type& x) {
    if ( IsEmpty ( ) ) return 0;
    x = elements[( front+1) % size];
    return 1;
}
```

队列的链表存储表示：链式队列



- 队头在链头，队尾在链尾
- 链式队列在进队时**无队满问题**，但**有队空问题**
- 队空条件为**front==NULL**

链式队列的类定义

```
template <class Type> class Queue;
template <class Type> class QueueNode {
    friend class Queue<Type>;
    private:
        Type data;  // 队列结点数据
        QueueNode<Type> *link;  // 结点链指针
    public:
        QueueNode ( Type d = 0, QueueNode<Type>
            *next = NULL ) : data (d), link (next) { }
};
```

链式队列的类定义

```
template <class Type> class Queue {
    private:
        QueueNode<Type> *front, *rear;  // 队头、队尾指针
    public:
        Queue ( ) : rear ( NULL ), front ( NULL ) { }
        ~Queue ( );
        void EnQueue (Type x);
        int DeQueue (Type& x);
        int GetFront (Type& x);
        void MakeEmpty ( );  // 实现与~Queue( )相同
        int IsEmpty ( ) { return front == NULL; }
};
```

链式队列的类定义

```
template <class Type> Queue<Type> :: ~Queue ( ) {
    QueueNode<Type> *p;
    while ( front != NULL ) {  // 逐个结点释放
        p = front;  front = front->link;  delete p;
    }
}
template <class Type> void Queue<Type> :: EnQueue (Type x) {
    if ( front == NULL )  // 创建第一个结点
        front = rear = new QueueNode <Type> (x, NULL);
    else { // 队列非空，插入
        rear->link = new QueueNode <Type> (x, NULL);
        rear = rear->link;
    }
}
```

链式队列的类定义

```
template <class Type> int Queue<Type> :: DeQueue (Type& x) {
    if ( IsEmpty ( ) ) return 0;  // 判队空
    QueueNode<Type> *p = front;
    x = front->data;  front = front->link;
    delete p;  return 1;
}

template <class Type> int Queue<Type> :: GetFront (Type& x) {
    if ( IsEmpty ( ) )  return 0;
    x = front->data;  return 1;
}
```

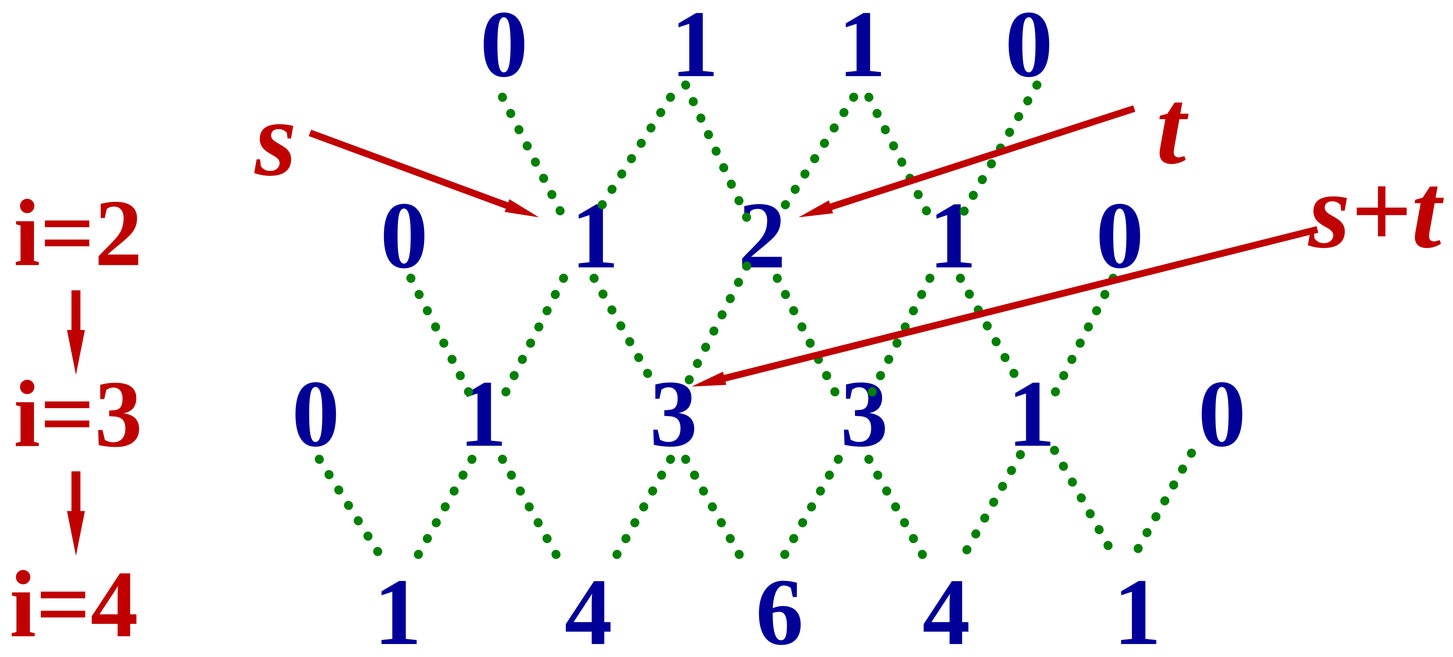
队列应用：逐行打印二项展开式$(a+b)^i$的系数

杨辉三角形(Pascal's Triangle)

			1		1				$i=1$	
		1		2		1				2
			1		3		3		1	3
		1		4		6		4	1	4
	1		5		10		10	5	1	5
1		6		15		20		15	6	1	6

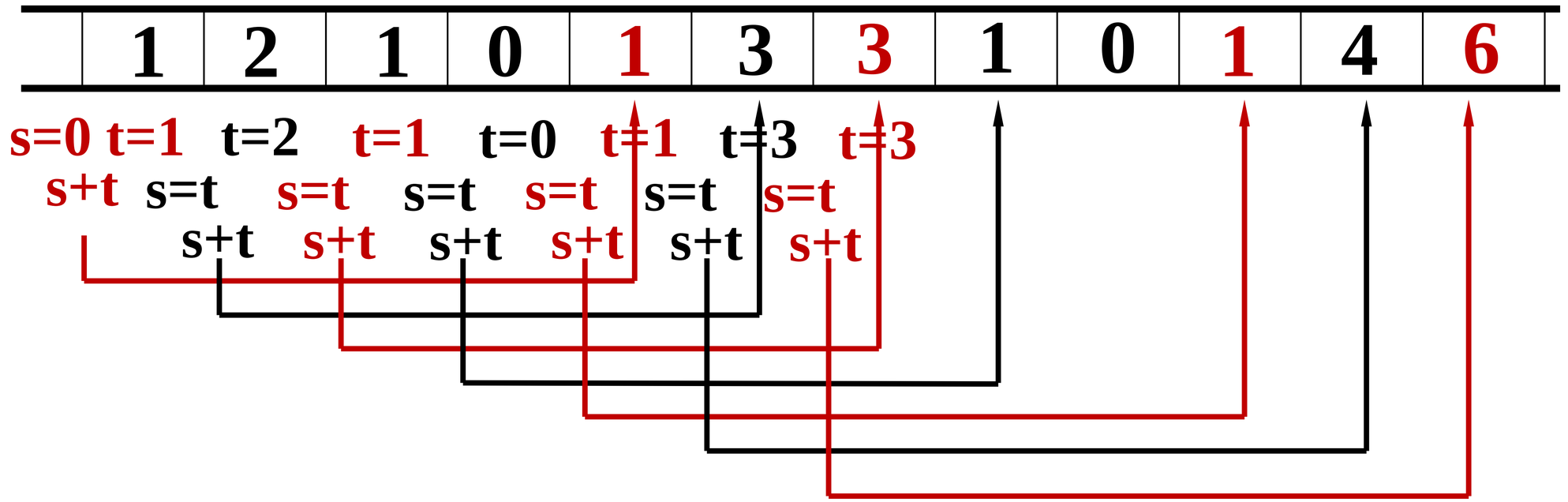
分析第i行元素与第i+1行元素的关系

两端加0



从前一行的数据可以计算下一行的数据

从第i行数据计算并存放第i+1行数据



i=2时，从q中取出t=1，计算s+t，得到i=3时第1个数据‘1’，顺序存放在q的后面。让s=t，再从数组中取出t=2，计算s+t，得到i=3时的第2个数据‘3’，顺序存放在q的后部……

一旦第i+1行数据形成，第i行的数据就不再放在数组q中

利用队列打印二项展开式系数的程序

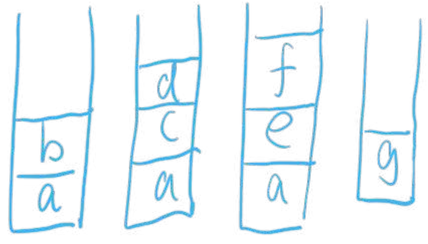
```
void YANGHUI (int n) {
    Queue q(n+3);  // 队列初始化：n+1+2
    int i = 1, j, s = 0, t, u;
    q.EnQueue(1);  q.EnQueue(1);  // 预先放入第一行的两个系数
    for ( int i = 1; i <= n; i++ ) {  // 逐行计算
        cout<<endl;
        q.EnQueue(0);  // 在各行系数之间插入一个0
        // 处理第i行的i+2个系数，生成第i+1行的系数
        for (int j = 1; j <= i + 2; j++) {
            q.DeQueue(t);  u = s + t;  q.EnQueue(u);  s = t;
            // 打印第i行的一个系数，第i+2个是0
            if (j != i + 2)  cout<<s<<‘ ’;
        }
    }
}
```

练习2

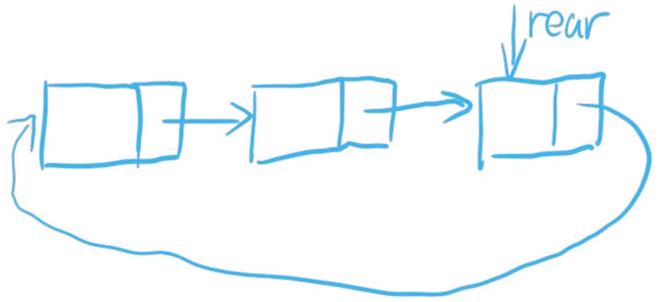
- 为解决计算机主机与打印机之间速度不匹配的问题，通常设置一个打印数据缓冲区。主机将要打印的数据依次写入缓冲区，而打印机则依次从缓冲区中取出数据。该缓冲区的逻辑结构应该是栈还是队列？
- 队列

- 设栈S和队列Q的初始状态均为空，元素a, b, c, d, e, f, g依次进栈S。如果每个元素出栈后立即进队Q，且出队的顺序为b, d, c, f, e, a, g，则栈S的容量至少是多少？
- 出栈的顺序为b, d, c, f, e, a, g，容量至少为3

出队的顺序 = 进队的顺序 = 出栈的顺序



练习2



- 以带头结点的循环链表表示队列，并且不设队头指针，只设队尾指针，试编写相应的初始化队、判队空、入队和出队的算法函数
- 队初始化：rear = new QNode(); rear->next = rear;
- 判队空：rear == rear->next;
- 入队：QNode * p = new QNode(); p->data = x; p->next = rear->next; rear = rear->next = p;
- 出队：若队非空，QNode * p = rear->next->next; rear->next->next = p->next; delete p;

练习2

- 利用两个栈来模拟一个队列，已知栈的三种运算定义为：Push(ST, x)、Pop(ST, x)、IsEmpty(ST)。利用栈的运算来实现该队列的三个运算：进队、出队及判队空
- 可以用两个栈模拟一个队列运算，即用一个栈作为输入(输入栈)，而另一个栈作为输出(输出栈)
  - 当进队时，总是将元素进栈到输入栈中
  - 当出队时，若输出栈已空，则将输入栈中元素全部出栈并进栈到输出栈中，然后从输出栈出栈数据；若输出栈非空，则从输出栈出栈数据
  - 只有在输入栈和输出栈都为空时，队列才为空

双向队列(Double-Ended Queue)

- **双向队列**是允许在两端插入和删除的线性表
- 双向队列没有队首和队尾之分，可用**两个指针left和right**分别指向左端和右端结点
- 双向队列空的标志为**left>right**
- 为了使得左右两端在一开始都能进行插入，取中间位置**m=(size–1)/2，并置left=m+1，right=m作为初始状态**
- **左端满的标志为left=0，右端满的标志为right=size–1**

双向队列的变型

- 在一端满而另一端不满的情况下，为了使得在满的一端能插入新结点，可以把整个队列向不满的一端移动
  - 如要避免整个队列的移动，还可把存放双向队列的数组首尾元素连接起来，构成环形双向队列
- 如固定双向队列的一端不动，插入和删除都在另一端进行，这样的双向队列就是一个栈
- 如果双向队列一端可以进行插入和删除，而另一端只能进行删除，这样的双向队列称为限制输入的双向队列
- 如果双向队列一端可以进行插入和删除，而另一端只能进行插入，这样的双向队列称为限制输出的双向队列

优先级队列(Priority Queue)

- 优先级队列：每次选择队列中具有最高优先级的元素出队，若具有最高优先级的元素同时有多个时，按它们在队列中的先后顺序出队
- 如下表：任务优先权及执行顺序的关系

任务编号	1	2	3	4	5
优先权	20	0	40	30	10
执行顺序	3	1	5	4	2

数字越小，优先权越高

优先级队列的类定义

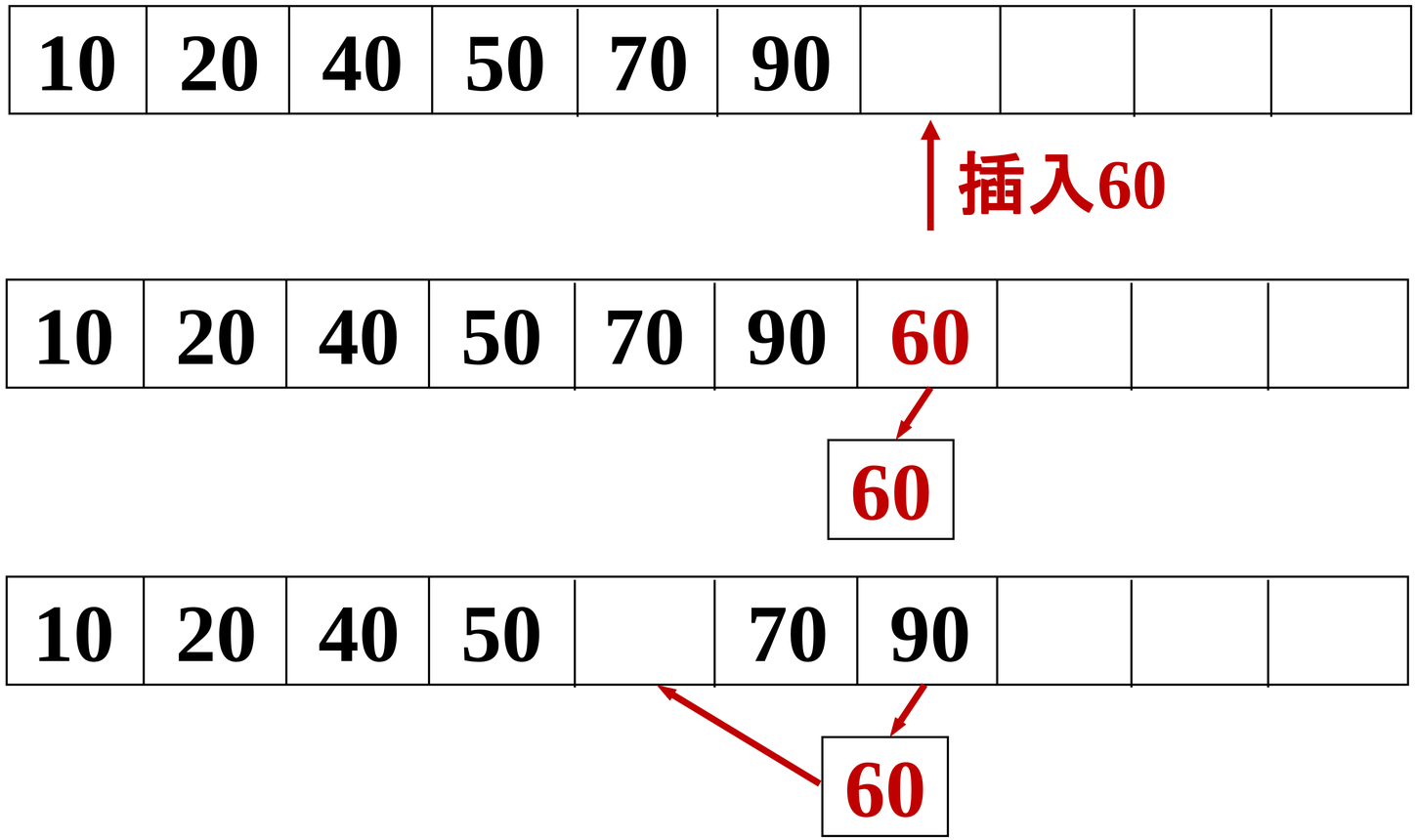
```
template <class Type> class PQueue {
  private:
    Type * pqelements;  // 存放数组
    int count;  // 当前队列元素个数
    int maxSize;  // 最大队列元素个数
  public:
    PQueue(int sz=50);
    ~PQueue( ) { delete [ ] pqelements; }
    void EnQueue(Type x);  // 入队
    int DeQueue(Type &x);  // 出队
    int GetFront(Type &x);
    void MakeEmpty( ) { count=0; }
    int IsEmpty( ) const { return count == 0; }
    int IsFull( ) const { return count == maxSize; }
};
```

优先级队列部分成员函数的实现

```
template <class Type> PQueue<Type> :: PQueue (int sz) {
    maxSize = sz;  count = 0;
    pqelements = new Type[maxSize];
    assert ( pqelements != NULL );
}
```

```
template <class Type> void PQueue<Type> :: EnQueue (Type x) {
    assert ( !IsFull ( ) );
    for ( int j = count−1; j >= 0; j−− )  // 从后向前找插入位置
        if ( pqelements[j] <= x ) break;
        else pqelements[j+1] = pqelements[j];
    pqelements[j+1] = x;
    count++;
}
```

从后向前找插入位置


10 20 40 50 70 90
插入60
10 20 40 50 70 90 60
60
10 20 40 50 70 90 60
60
10 20 40 50 60 70 90

优先级队列部分成员函数的实现

```
template <class Type> int PQueue <Type> :: DeQueue(Type &x) {
    if (count == 0)  return 0;
    x = pqelements[0];  // 取出0号元素
    for ( int i = 1; i < count; i++ )
        pqelements[i-1] = pqelements[i]; // 从前向后移动元素填补空位
    count--;
    return 1;
}

template <class Type> int PQueue<Type> :: GetFront (Type& x) {
    if ( IsEmpty ( ) )  return 0;
    x = pqelements[0];
    return 1;
}
```

本章小结

- 栈
- 队列

作业

- n个元素依次进栈，则可能的出栈序列共有多少？

- 写出以下表达式的后缀表达式。
  - a*(b+c)-d
  - (a+a*b)*a+c*b/a
  - a+((b*c-d)/e+f*g/h)+i/j

- 利用栈实现单链表上的逆置运算。

- 设用链表表示一个双向队列，要求可以在表的两端插入，但只能在表的一端删除，试编写该队列的入队、出队、及判队空的算法。